Entrega II Algoritmo e logica de programação

Player

Objetivo:

Algoritmo de movimentação do personagem, objetivo reconhecer o input das teclas WASD para movimentar o personagem conforme o jogador desejar, permite que o personagem gire automaticamente na direção em que está se movimentando.

Código:

```
using UnityEngine;

public class PlayerMovement : MonoBehaviour
{
    [Header("Configurações de Movimento")]
    public float velocidade = 5f;
    public float velocidadeRotacao = 10f;

    private Rigidbody rb;
    private Vector3 moveInput;

    void Start()
    {
        rb = GetComponent<Rigidbody>();
        rb.freezeRotation = true;
    }

    void Update()
    {
        if (DialogueManager.Instance != null && DialogueManager.Instance.EstaEmDialogo())
        {
            moveInput = Vector3.zero;
            return;
        }

        float moveX = Input.GetAxisRaw("Horizontal");
        float moveZ = Input.GetAxisRaw("Vertical");

        moveInput = new Vector3(moveX, 0f, moveZ).normalized;
    }

    void FixedUpdate()
    {
        if (moveInput.sqrMagnitude > 0)
        {
            Vector3 targetPosition = rb.position + moveInput * velocidade * Time.fixedDeltaTime;
            rb.MovePosition(targetPosition);
            Quaternion targetRotation = Quaternion.LookRotation(moveInput);
            rb.MoveRotation(Quaternion.Slerp(rb.rotation, targetRotation, Time.fixedDeltaTime * velocidadeRotacao));
        }
    }
}
```

Resultado esperado:

Com esse algoritmo o personagem consegue se movimentar livremente pelos comandos do jogador e rotacionar na direção que se movimenta.

Interação e dialogo com inimigo

Objetivo:

Esse algoritmo tem como objetivo iniciar um dialogo com o player assim que ele entra em uma área de interação do personagem inimigo, assim ele reconhece e começa uma sequência de diálogos.

Código:

```
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.Events;

public class EnemyDialogueTrigger : MonoBehaviour
{
    [Header("Configurações do Diálogo")]
    public List<DialogueLine> dialogoInimigo;
    public bool dispararApenasUmaVez = true;

    [Header("Eventos Opcionais")]
    public UnityEvent aoConcluirDialogo;

    private bool jaDisparou = false;

    private void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag("Player"))
        {
            if (dispararApenasUmaVez && jaDisparou) return;
            jaDisparou = true;
            if (DialogueManager.Instance != null)
            {
                DialogueManager.Instance.IniciarDialogo(dialogoInimigo, aoConcluirDialogo);
            }
        }
    }
}
```

Resultado esperado:

Quando o jogador se aproximar de um inimigo que entra em sua área de interação, o jogo inicia automaticamente o dialogo configurado. Caso a opção execução única esteja ativa a conversa não inicia novamente após ter sido disparada.

Mudar Cena

Objetivo:

Esse algoritmo tem como objetivo permitir a transição entre diferentes cenas do jogo possibilitando que o jogador avance para uma nova fase.

Código:

Arquivos Diversos

```
1   using UnityEngine;
2   using UnityEngine.SceneManagement;
3
4   public class MudarCena : MonoBehaviour
5   {
6       public void CarregarCena(string Fase_1)
7       {
8           SceneManager.LoadScene("Fase_1");
9       }
10  }
```

Resultado esperado:

Ao executar o método CarregarCena, o sistema carrega a cena Fase_1 permitindo que o jogador avance para a cena correspondente.

Algoritmo de acompanhamento do jogador

Objetivo:

Esse algoritmo permite que a tortuguita acompanhe o jogador automaticamente,

o sistema calcula a distância um limite mínimo para acompanhar e começar a se movimentar, além de controlar a velocidade da tortuguita e sua rotação.

Código:

```
using UnityEngine;

public class TortuguitaFollower : MonoBehaviour
{
    [Header("Alvo a Seguir")]
    public Transform player;

    [Header("Configurações de Acompanhamento")]
    public float velocidade = 4f;
    public float distanciaMinima = 2f;
    public float velocidadeRotacao = 10f;

    void Start()
    {
        if (player == null)
        {
            GameObject playerObj = GameObject.FindGameObjectWithTag("Player");
            if (playerObj != null)
            {
                player = playerObj.transform;
            }
        }
    }

    void Update()
    {
        if (player == null) return;
        if (DialogueManager.Instance != null && DialogueManager.Instance.EstaEmDialogo()) return;
        Vector3 alvoMesmaAltura = new Vector3(player.position.x, transform.position.y, player.position.z);
        float distancia = Vector3.Distance(transform.position, alvoMesmaAltura);

        if (distancia > distanciaMinima)
        {
            Vector3 direcao = (alvoMesmaAltura - transform.position).normalized;
            transform.position = Vector3.MoveTowards(transform.position, alvoMesmaAltura, velocidade * Time.deltaTime);
            if (direcao != Vector3.zero)
            {
                Quaternion rotacaoDesejada = Quaternion.LookRotation(direcao);
                transform.rotation = Quaternion.Slerp(transform.rotation, rotacaoDesejada, velocidadeRotacao * Time.deltaTime);
            }
        }
    }
}
```

Resultado esperado:

Após iniciar a fase a tortuguita identifica o personagem principal e passa a acompanha-lo automaticamente pelo cenário, respeitando o limite da distancia mínima.